

CE 27 Softwaretechnik

Prof.Dr. Thomas Baar

thomas.baar@htw-berlin.de



**Hochschule für Technik
und Wirtschaft Berlin**

University of Applied Sciences

Sommersemester 2026

Vorlesung

Heutiges Thema

Anforderungserhebung (engl.: Requirement Elicitation)
mit Anforderungsfällen (engl. Use Cases)

Problem: Auftraggeber haben im Allgemeinen nur eine **vage Vorstellung** davon, was die von Ihnen in Auftrag gegebene Software unter welchen Umständen leisten soll.

Lösung: **Use Cases** sind eine **Notation**, um **funktionale Eigenschaften strukturiert** darzustellen

Zusätzliche Literatur

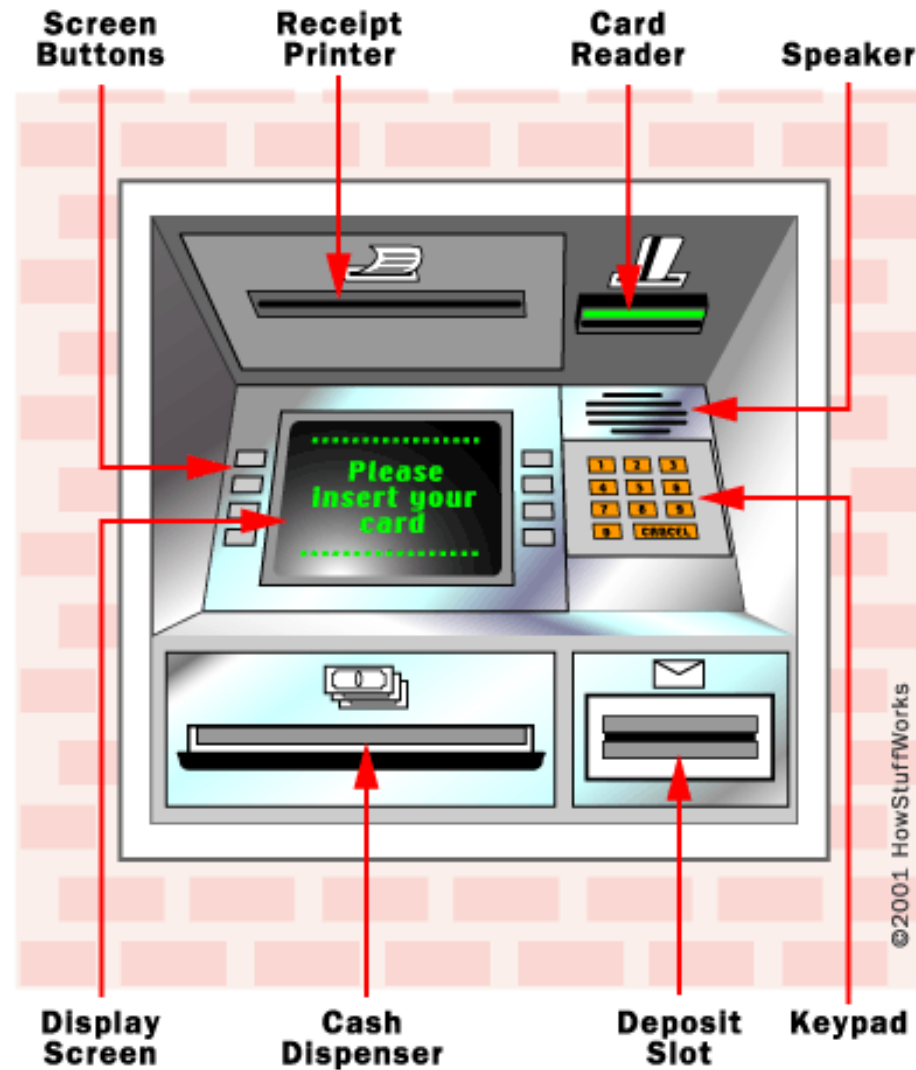
- Ivar Jacobson: *Use cases - Yesterday, today, and tomorrow*; *Journal on Software and Systems Modeling*; 3:210-220; 2004.
 - Über die **Historie** von Use Cases von dessen Erfinder
- Ivar Jacobson, Ian Spence, Brian Kerr: *Use-Case 2.0*; *Communications of the ACM*, 59-05, 61-69, May 2016.
 - Neues Konzept "Use-Case slice" eingeführt
- Alistair Cockburn: *Writing Effective Use Cases*; Addison-Wesley, 2001.
 - Einflussreiches Buch über die **Kunst des Formulierens** von Use Cases

Einführungsbeispiel Geldautomat

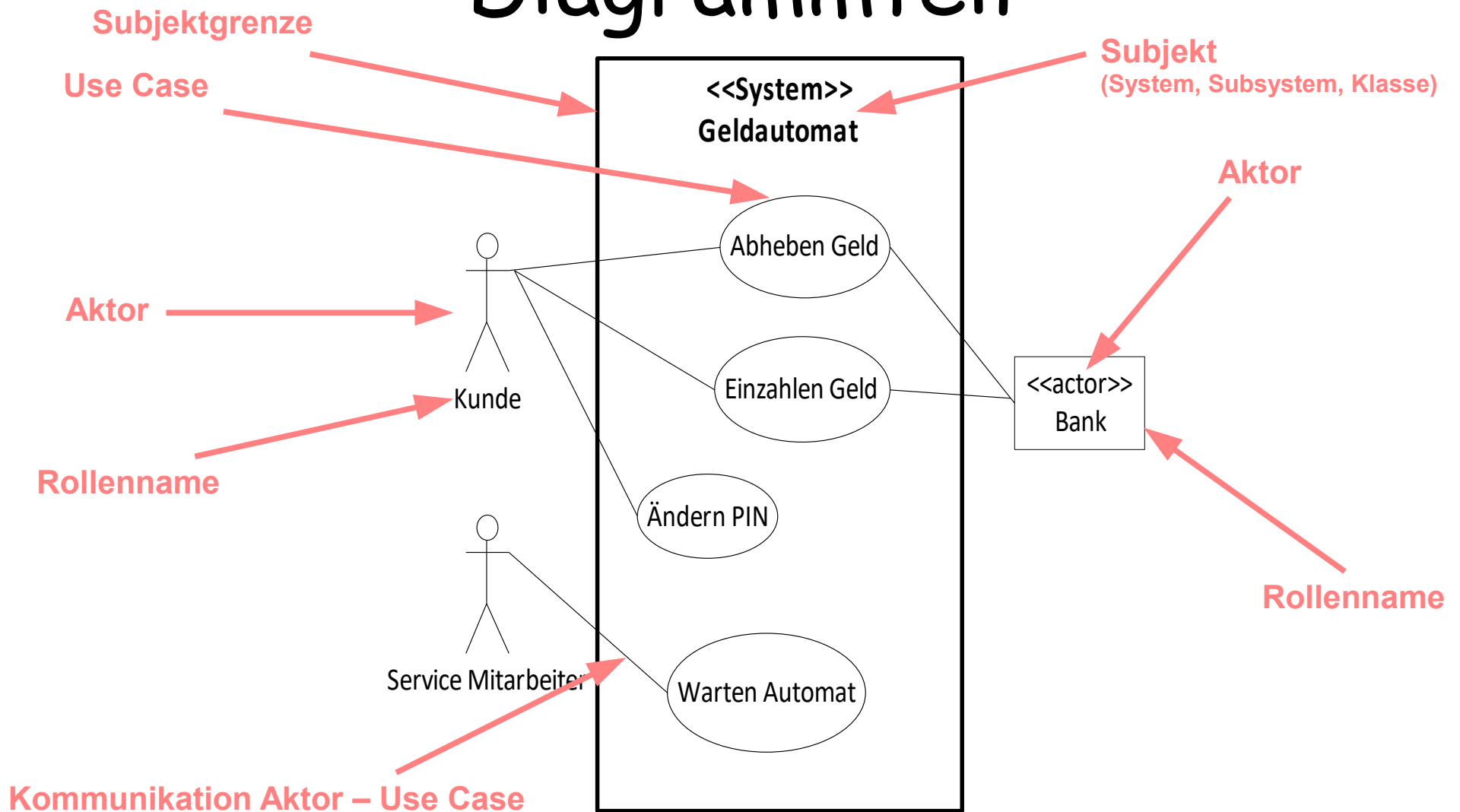


Wie kann **Funktionalität** beschrieben werden?

Einführungsbeispiel Geldautomat



Use Case Modell - Diagrammteil



Use Case Modell - Diagrammteil

Der Diagrammteil gibt einen Überblick über den Kontext des Subjekts und über die Interaktionen zwischen Aktoren und Subjekt. Ein UC Diagramm identifiziert die gewünschte Funktionalität (Use Cases) und alle beteiligten Aktoren.

Use Case - Repräsentiert die Interaktion zwischen Akteur und Subjekt. UCs sind mit dem Ziel des Primären Aktors beschriftet.

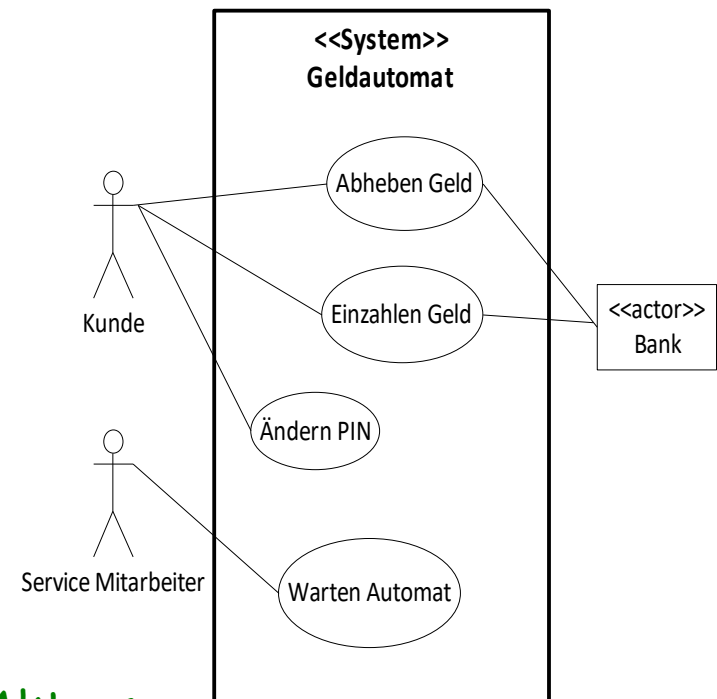
Akteur - *Primärer Akteur*: Benutzt das Subjekt, um ein bestimmtes Ziel zu erreichen
Sekundärer Akteur: Wird vom Subjekt benutzt.

Subjektgrenze - Legt die Grenze zwischen Subjekt und Kontext (Umgebung) fest

Use Cases

Ein Use Case beschreibt, wie ein Akteur das Subjekt für etwas Wichtiges(!) benutzt, d.h. wie der Akteur sein Ziel erreicht.

- Identifizierung von Use Cases
 - In der Anfangsphase des Projekts (Spezifikation)
 - Entwickler sollte Nutzer und andere Beteiligte (Stakeholder) interviewen
 - richtige Granularität wählen (Anz. < 40)
- Mehr als ein Akteur kann an UC teilnehmen
- Use Case besteht aus zwei Teilen:
 - Diagrammteil zeigt lediglich Ziel des Primären Aktors an
 - Textueller Teil besteht aus Ereignisfluss (Flow of Events) von Interaktionen zwischen Akteur und Subjekt, so dass Ziel erreicht wird



Use Case -- Benennungen

- **Aktive Verbphrase** (Abheben Geld, Einzahlen Geld,...)
- **In der Sprache des Nutzers**

Nuttersprache

- *"Formatiere Dokumente in konsistenter Weise"*
- *"Formatiere zwei Dokumente im gleichen Stil"*

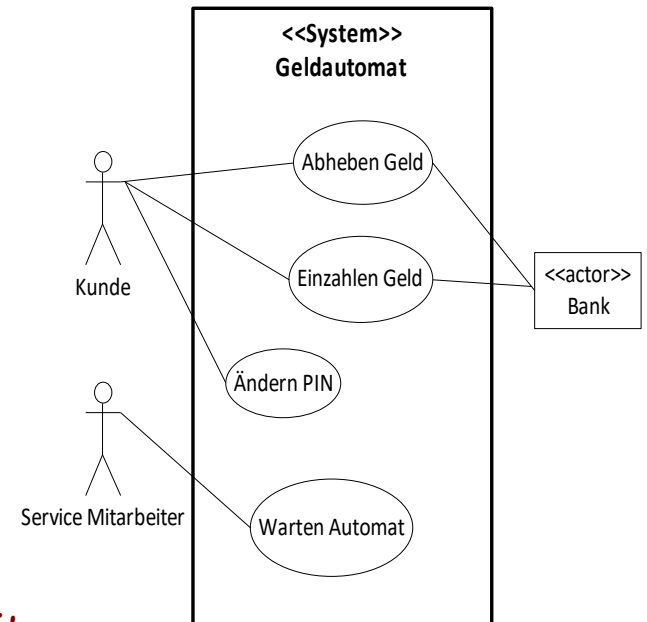
Technische Sprache

- *"Definiere/Wende an/Ändere einen Stil "*
- *"Importiere einen Stil"*
- *"Wende Stil an"*

Aktor

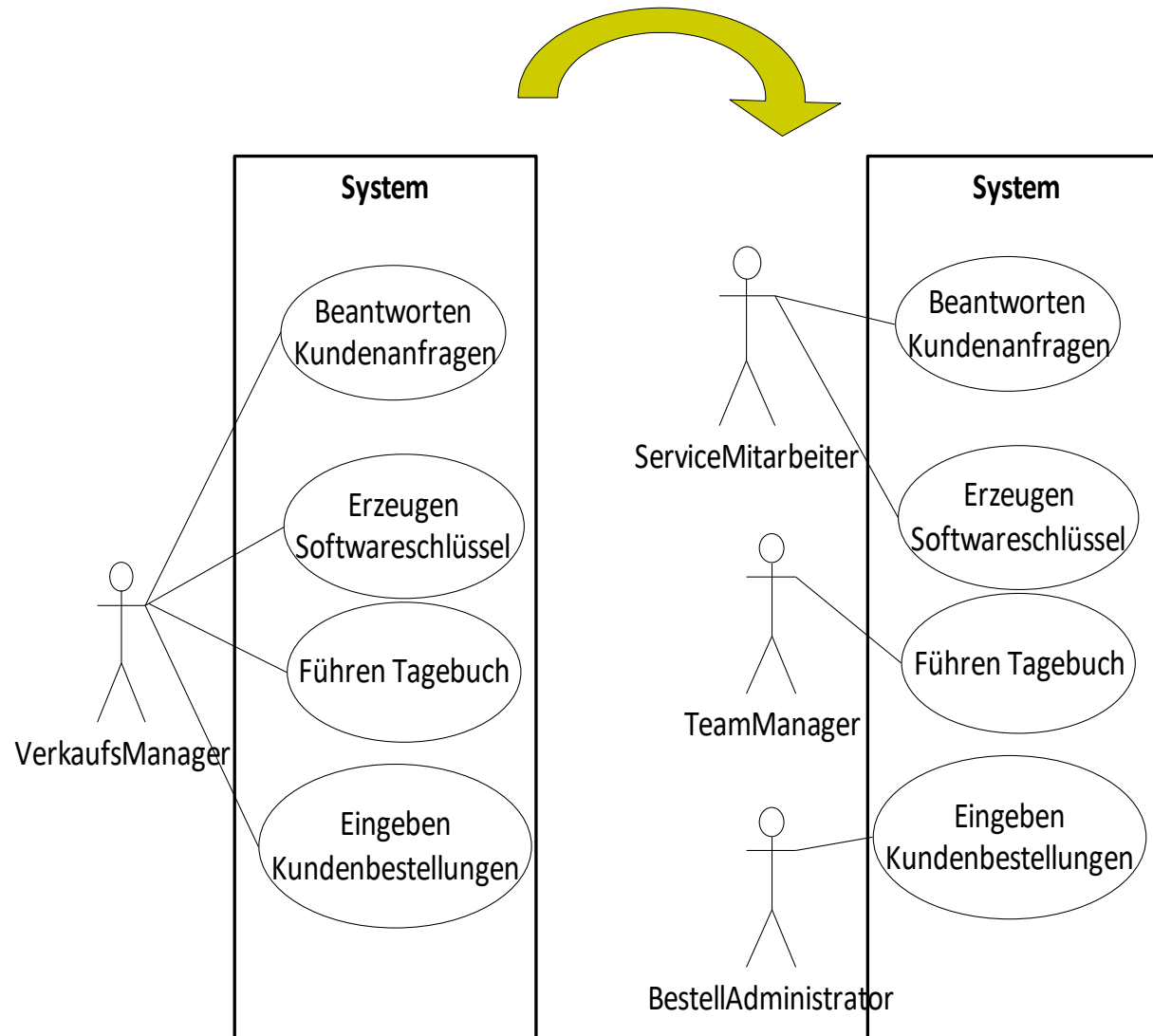
Repräsentiert Nutzer, der mit Subjekt in bestimmter **Rolle** interagiert

- Typische Rollen sind:
 - *Kunde, Anrufer, Angerufene*
 - *Manager, Handler, Administrator, Service Mitarbeiter*
 - *Banksystem, Authentifizierungssystem, Abrechnungssystem*
- Kann eine Person, Personengruppe, externes System repräsentieren
- Derselbe Nutzer kann mehrere Rollen haben
- Unterscheidung in
 - **Primärer Aktor** - hat ein Ziel gegenüber Subjekt
 - **Sekundärer Aktor** - Wird vom Subjekt genutzt

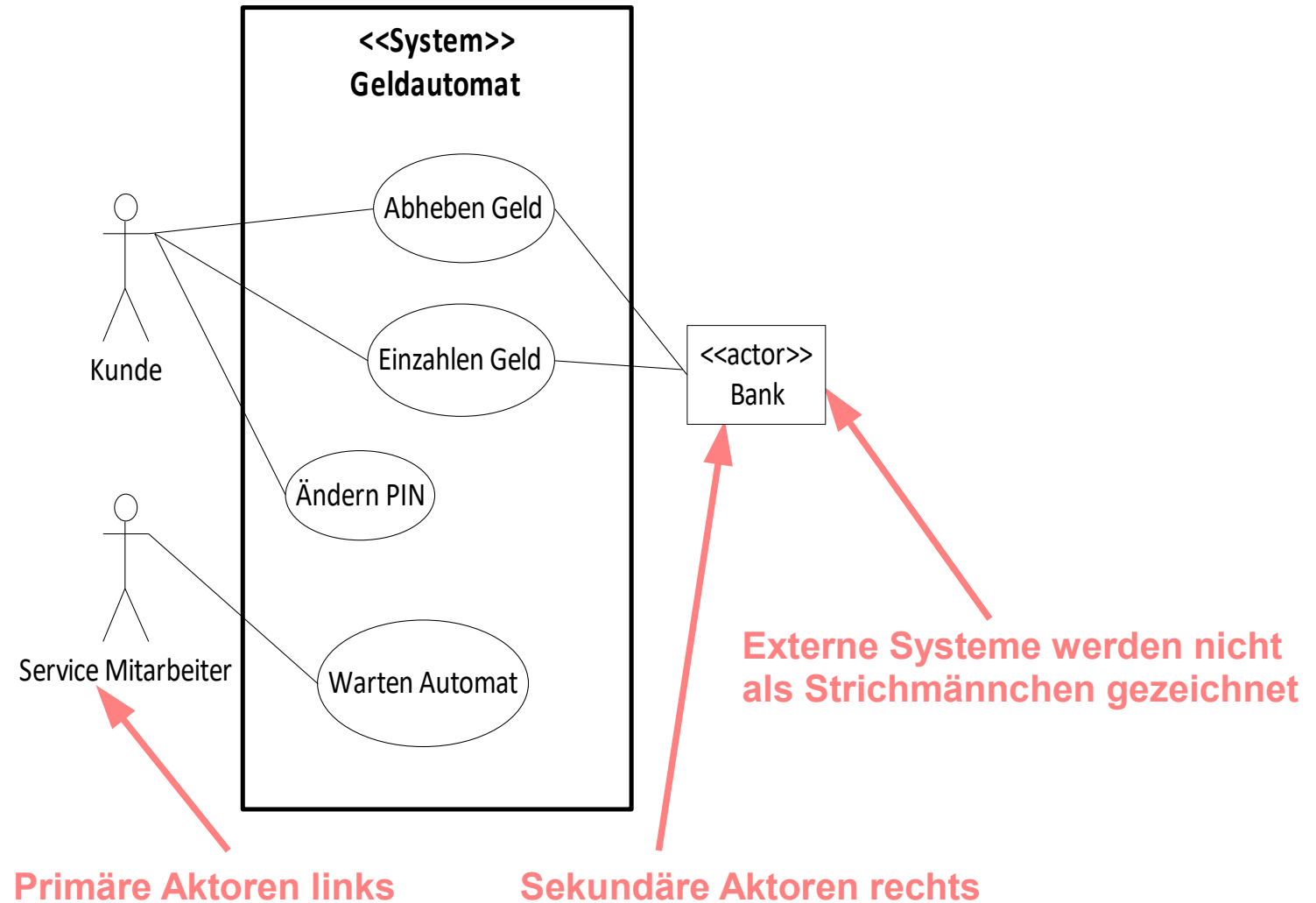


Aktoren und Ziele

Unterschiedliche Akteure erleichtern das Finden von Use Cases



Layout von UC Diagrammen



UC Modell - Textueller Teil

Textueller Teil (Story) - Abfolge von Ereignissen (flow of events), wobei ein Ereignis das zeitlich lokalisierbare Auftreten einer wichtigen Sache ist

- Textueller Teil macht 90% (!) eines Use Case aus
- Ereignistypen
 - **Nutzeraktion** (*Kunde führt Karte ein, Anrufer nimmt Hörer ab*)
 - **Antwort externes System** (*Authentifizierungssystem validiert PIN*)
 - **Zeitsteuerung** (*Am Ende des Monats...*)
- Eine Story besteht aus Menge von Szenarien, und zwar aus
 - Einem Basisszenarium (aka "Happy Day Scenario")
 - Mehreren alternativen Szenarien für optionales Verhalten und Ausnahmen/Fehler
 - Erfahrungswert: 60-80% von Software für Ausnahmebehandlung

Szenarium - Beispiel

EinzahlenGeld (Happy Day Scenario)

- 1) Der **Kunde** **führt** seine Bankkarte in den Geldautomat **ein** und **gibt** seine Personal Identification Number (PIN) **ein**.
- 2) Das **System** **validiert** die PIN.
- 3) Der **Kunde** **äussert** den **Wunsch** nach Geldeinzahlung **und führt** alle Banknoten des einzuzahlenden Betrages **ein**.
- 4) Das **System** **prüft** die Banknoten **und zeigt** den Betrag der Einzahlung **an**.
- 5) Der **Kunde** **bestätigt** den angezeigten Betrag.
- 6) Das **System** **informiert** die **Bank**, den eingezahlten Betrag auf das Konto des Kunden zu buchen.
- 7) Das **System** **gibt** die Karte **aus**.
- 8) Der **Kunde** **nimmt** die Karte.
- 9) Das Use Case endet erfolgreich.

Szenarium - Generelle Struktur

- 1) Der Primäre Aktor macht eine Eingabe beim System (evtl. mit Daten).
- 2) Das System validiert Eingabe und Daten.
- 3) Das System ändert seinen internen Zustand und sendet gegebenenfalls Eingaben zu Sekundären Aktoren.
- 4) Das System antwortet dem Primären Aktor mit einem Resultat.

UC Modell - Tipps für die Praxis

- Bei Projektbeginn detailliert entwickeln und später revidieren
- Mit anderen Artefakten ergänzen
 - Glossar beinhaltet informelle Beschreibung der Domänenterminologie
 - Domänenmodell
- Erlaubt systematische, effektive Kommunikation mit allen Beteiligten (Stakeholdern)
 - Gibt es Alternativen bei einem Ereignis?
 - Ist das formulierte Ziel wirklich das Ziel, welches der Akteur hat?
- Sehr geeignet, um Ziele von anderen Beteiligten zu verstehen
- Erlaubt vereinfachte/abstrakte Sicht auf das System

Glossar

- Beschreibt die **Hauptbegriffe** einer Domäne (eines Fachgebiets)
- Ist in **natürlicher Sprache** verfasst
- **Erleichtert den Einstieg** neuer Projektmitglieder
- Verhindert längliche Beschreibungen
- **Verhindert redundante** Beschreibungen
 - Begriff wird im Glossar definiert und kann mit klarer Bedeutung genutzt werden

Glossar - Beispiel

"Haltung --

Eine Haltung ist die Verbindungstrecke eines Abwasserkanales zwischen zwei Schächten und dient der Ableitung von Abwasser .

An die Haltungen werden die Anschlussleitungen in Fließrichtung vorgenommen.

Kanalhaltungen sollen gerade und ohne Gefällewechsel verlegt werden.

Richtungs-, Gefälle- und Dimensionswechsel/Änderungen sollen nur an den Schächten erfolgen, was aber nicht immer möglich ist.

Eine Haltung besteht aus mehreren ineinander gesetzten Rohren diverser Materialien, z.B. Steinzeug, Kunststoff, Beton o.ä."

Quelle: <http://www.kanalinspekteure.de>

UC Formulierungstipps

Konsistente Formulierungen (Granularität, Stil) machen das Leben einfacher (sowohl für Verfasser als auch für Leser).

- Einfache Satzstruktur: Subjekt + Verb + Objekt
 - Subjekt ist Akteur/System, welcher Aktion anstößt/kontrolliert
 - *Das System zieht den Betrag vom Kontoguthaben ab.*
- Klar machen wer den Ball hat ("who has the ball")
 - Eine Story ist ähnlich wie Fußball spielende Kinder (Akteuren/System), die Ball kicken (Nachrichten/Daten werden von Akteur zu Akteur gegeben)
- Aus Vogelperspektive (be)schreiben und nicht aus Programmiersicht
 - *Der Kunde gibt Karte und PIN in den Geldautomat ein. 😊*
 - *Gib Karte und PIN. ☹*

UC Formulierungstipps

- Richtige Granularität der Aktionen
 - Warum macht der Akteur dies?
 - Nutzer gibt Name und Adresse ein 😊
 - Nutzer berührt eine Taste ☹️
 - Cockburn: Gute Happy Day Szenarios haben höchstens 9 Schritte.
- Intention(Absicht) des Akteurs, nicht dessen Arbeitsschritte beschreiben
 - Nutzer gibt Name und Adresse ein 😊
 - 1. System fragt nach Name, 2. Nutzer gibt Name ein, 3. System fragt nach Adresse, 4. Nutzer gibt Adresse ein und klickt of "ok" ☹️

UC Formulierungstipps

Ein Schritt in der Realität ist ein Schritt in der Beschreibung:

- 1. Der Kunde *gibt* die Bestellnummer *ein*.
- 2. Das System *stellt fest*, dass es die Nummer des Monatsgewinns ist.
- 3. Das System *registriert* den Kunden als den Monatsgewinner *und informiert* den VerkaufsManager.
- 4. Das System *gratuliert* dem Kunden *und gibt* Informationen, wie der Gewinn in Empfang genommen werden kann.

Anstatt

- 1. Der Kunde gibt die Bestellnummer ein. Das System stellt fest, dass es die Nummer des Monatsgewinns ist. Das System registriert den Kunden als den Monatsgewinner und informiert den VerkaufsManager. Das System gratuliert dem Kunden und gibt Informationen, wie der Gewinn in Empfang genommen werden kann.

UC Formulierungstipps

- 'Validieren' ist besser als 'Prüfen ob'
 - 'Prüfen ob' kommt im Szenario keinen Schritt weiter
 - Man muss danach immer schreiben "Wenn die Prüfung erfolgreich war ..." "Wenn die Prüfung fehlschlug ..."
 - Bessere Wörter: 'Validieren', 'Verifizieren', 'Sicherstellen'
 - 1. Das System *verifiziert* das Passwort als korrekt. 😊
 - 2. Das System fragt die Datenbank nach dem Nutzerprofil.

Besser als

- 1. Das System *prüft, ob* das Passwort korrekt ist. ☹️
- 2. Bei korrektem Passwort fragt das System nach dem Nutzerprofil.

UC Formulierungstipps

Idiom: "Mache Schritte x - y bis Bedingung gilt"

Manchmal müssen ein Schritt oder Gruppe von Schritten wiederholt werden.

- Kann im Schritt selbst formuliert werden
 - Der Nutzer *wählt* ein oder mehrere Produkte *aus*.
 - Der Nutzer *sucht* in den Produkten *bis* er das Passende gefunden hat.
- Kann über Meta-Informationen formuliert werden
 - 1. Der Kunde *gibt* entweder Kundennummer oder Name und Adresse *ein*.
 - 2. Das System *zeigt* die gewählten Einstellungen des Kunden *an*.
 - 3. Der Kunde *wählt* einen Artikel zum Kauf.
 - 4. Das System *fügt* den Artikel dem Einkaufskorb des Kunden *hinzu*.
 - Kunde *wiederholt Schritt 3. und 4. bis* er angibt, den Einkauf beenden zu wollen.
 - 5. Kunde *kauft* die Artikel aus dem Einkaufskorb.

UC Formulierungstipps

- Vermeidung von 'sollte'/'könnte', die Aussagen abschwächen
 - Das System validiert den eingegebenen Betrag. 😊
 - Der eingegebene Betrag **sollte** vom System validiert werden. ☹
- Vermeidung vager Ausdrücke 'sehr', 'mehr', 'eher', 'ausreichend', 'relevant',
- Zeitform immer Präsens (keine 'wird')
- Verwendung von 'wenn' nur in Bedeutung von "Wenn <Bedingung> dann <Aktion> sonst skip"
 - Verzweigungen im Szenario werden gesondert behandelt

Komplexere UseCases

Vorlage (**Template**) für UC-Beschreibungen

Für die **textuelle Beschreibung** eines Use Case hat sich folgende Vorlage bewährt:

Use Case: **Name** des Use Case

Kurzbeschreibung: Eine **Zusammenfassung** des Use Case

Vorbedingung: Welche **Annahmen** können vorausgesetzt werden, wenn das **Use Case** beginnt

Nachbedingung: Welche Eigenschaften **gelten immer** im Endzustand eines jeden Szenarios (**worauf können sich Akteure verlassen**)

Primärer Akteur: Akteur, der das Use Case **anstößt** (engl. triggered)

Erfolgsszenario: Beschreibung der normalen, erwarteten Folge von Interaktionen (Interaktionsfolge, die **von den meisten Nutzern in den meisten Fällen** gewählt wird, aka Happy Day Szenario)

Erweiterungen: Beschreibung von **alternativem** und **optionalem** Verhalten

Nummerierungsschema der Schritte

Die Verhaltensbeschreibungen unter *Erweiterungen* beziehen sich immer auf bestimmte Schritte im *Happy Day Szenario*. Zu diesem Zweck muss jeder Schritt eine eindeutige Nummer erhalten. Die Nummern werden nach einem bestimmten Schema vergeben.

- Jeder Schritt ist nummeriert durch eine Folge von Zahlen, Buchstaben oder sonstigen Symbolen (z.B. 5||b.2.) Das letzte Element ist immer ein Punkt ('.'), welcher nicht zur Nummer selbst gehört

- 1. Der Kunde führt die Karte ein.

Anstatt von

- 1 Der Kunde führt die Karte ein.

- Aufeinander folgende Nummern bedeuten, dass die Schritte einer nach dem anderen ausgeführt werden
 - die Reihenfolge kann jedoch explizit als unerheblich erklärt werden ("Schritt 1-4 erfolgen in beliebiger Reihenfolge")
 - 1. Der Kunde führt die Karte ein.
 - 2. Das System validiert den Kartentyp.

Nummerierung in Erweiterungen

Schritte in Erweiterungen beschreiben **optionales oder alternatives Verhalten**. Die Schritte und deren Nummerierung lassen sich in 3 Gruppen kategorisieren:

1. **Alternatives Ereignis**: <AktuelleNummer> <Buchstabe>

- 3. Der Kunde wählt ein Produkt aus.  Schritt aus Happy Day Szenario
- 3a. Der Kunde beendet Produktauswahl.  Schritt in Erweiterung
(Nummer 3a kennzeichnet Schritt als alternatives Ereignis)

(wenn Zweig 3a. genommen wird, ist Ereignis 3. **nicht** aufgetreten!)

Nummerierung in Erweiterungen

2. **Zusätzliches** Ereignis bei bestimmter Bedingung (durch Doppelpunkt markiert): *<AktuelleNummer> "||" <Buchstabe>*

Doppelpunkt
markiert Bedingung

- 3. Der Kunde kauft ein Produkt.
- 3||a. Das System stellt fest, dass der Kunde sich für das Bonus-Programm qualifiziert hat:
- 3||a.1. Das System fügt 10 Bonuspunkte dem Kundenkonto hinzu.

(wenn Bedingung 3||a gilt, dann tritt Ereignis 3||a.1 zusätzlich zu Ereignis 3 auf, danach geht es mit Schritt 4 weiter)

Nummerierung in Erweiterungen

3. **Sequentielle** Ereignisse: <Aktuelle Nummer> "." <Zahl>

- *3||a. Das System stellt fest, dass der Kunde sein Konto überzogen hat:*
- *3||a.1. Das System informiert den Kunden über die Re-Initialisierung des Warenkorbs.*
- *3||a.2. Use Case wird mit Schritt 2 fortgesetzt.*

Ereignisse 3||a.1, 3||a.2 treten in dieser Reihenfolge auf.

Beispiel: UC KaufeGetränk

Use Case: KaufeGetränk

Kurzbeschreibung : Der Käufer möchte ein Getränk von einem Verkaufsautomaten kaufen. Das bedeutet den Austausch einer bestimmten Geldsumme für ein Getränk.

Primärer Akteur: Käufer

Vorbedingung: Jedem Getränk ist ein Preis zugeordnet.

Nachbedingung: Falls ein Getränk verkauft wird, so ist sein Preis genau die Höhe der Geldsumme, die der Automat vom Käufer erhalten hat.

Beispiel: UC KaufeGetränk (Forts.)

Erfolgsszenario:

1. Der Käufer **wirft eine Münze** in den Automaten.

Schritt 1 kann so oft wiederholt werden wie der Käufer wünscht.

2. Käufer **wählt ein Getränk** aus.

3. Das System **validiert**, dass das eingeworfene **Geld** zum Kauf des gewählten Getränks **ausreichend** ist und gibt den **überzahlten Betrag** zurück.

4. Das System **gibt das Getränk** aus.

5. Use Case endet erfolgreich.

Beispiel: UC KaufeGetränk (Forts.)

Erweiterungen:

(1-2)a. Der Käufer **verlangt** die **Rückgabe** des eingesteckten Münzen.

(1-2)a.1. Das System **gibt** eingesteckte Münzen **aus**.

(1-2)a.2. Use Case endet erfolglos.

1||b. Das System ist **nicht funktionstüchtig**:

1||b.1. Das System **gibt** eingesteckte Münzen **aus**.

1||b.2. Use Case endet erfolglos.

Beispiel: UC KaufeGetränk (Forts.)

Erweiterungen:

3a. System **stellt fest**, dass das **eingeegebene Geld** zum Kauf des gewählten Getränks **nicht ausreichend** ist.

3a.1. System **informiert** Käufer.

3a.2. Use Case wird in Schritt 1 oder 2 fortgesetzt.

4a. System **stellt fest**, dass das gewählte Getränk **nicht mehr vorrätig** ist.

4a.1. System **informiert** Käufer.

4a.2. Use Case wird in Schritt 1 oder 2 fortgesetzt.

Beispiel: UC AuthentifiziereKunde

Use Case: AuthentifiziereKunde

Kurzbeschreibung : Der Kunde möchte sich beim System **authentifizieren**, was nur über eine Bankkarte und eine **Personal Identification Number (PIN)** möglich ist. Das System benutzt den Service eines **Bank Authentication Server (BAS)** zur Verifikation der PIN.

Primärer Akteur: Kunde

Beispiel: UC AuthentifiziereKunde (Forts.)

Erfolgsszenario:

1. Der Kunde **führt** die Karte **ein**.
2. Das System **liest** die Karte **aus** und **validiert Kartentyp***.
3. Der Kunde **gibt** PIN **ein**.
4. Das System **fordert** das BAS **auf**, die Informationen zur Kundenauthentifizierung zu **verifizieren**.*
5. Das BAS **informiert** das System, dass Authentifizierungsinformationen **valide sind**.
6. System **informiert** Kunde.
7. Use Case endet erfolgreich.

* -- Technische Terminologie ist im Glossar erläutert

Beispiel: UC AuthentifiziereKunde (Forts.)

Erweiterungen:

(1-6)a. (jederzeit) Der Kunde **bricht** die Authentifizierung **ab**.

(1-6)a.1. Das System **gibt** die Karte **aus**.

(1-6)a.2. Use Case endet erfolglos.

1||b. Das System ist **nicht funktionstüchtig**:

1||b.1. Das System **gibt** die Karte **aus** und **informiert** den Kunden, dass es zur Zeit **ausser Betrieb** ist.

1||b.2. Use Case endet erfolglos.

Beispiel: UC AuthentifiziereKunde (Forts.)

Erweiterungen:

2b. Das System **stellt fest**, dass der Kartentyp nicht unterstützt wird.

2b.1. Das System **informiert** den Kunden über unerwarteten Kartentyp und **gibt** die Karte **aus**.

2b.2. Use Case endet erfolglos.

3b. Das System **überschreitet** beim Warten auf die PIN-Eingabe ein **Zeitlimit**.

3b.1 Das System **informiert** den Kunden über das überschrittene Zeitlimit und **gibt** die Karte **aus**.

3b.2. Use Case endet erfolglos.

Beispiel: UC AuthentifiziereKunde (Forts.)

Erweiterungen:

5b. Das BAS informiert das System über inkorrekte Authentifizierungsinformationen.

5b.1. Das System stellt fest, dass der Kunde noch nicht zum dritten Mal eine falsche PIN eingegeben hat. Das System informiert den Kunden und fordert ihn zur erneuten Eingabe der PIN auf.

5b.2. Use Case wird in Schritt 3 fortgesetzt.

5b.1a Das System stellt fest, dass der Kunde zum dritten Mal eine falsche PIN eingegeben hat.

5b.1a.1. Das System zieht die Karte ein und informiert Kunden, sich an seine Bank zu wenden.

5b.1a.2. Use Case endet erfolglos.

Alternative!



Beispiel: UC AuthentifiziereKunde (Forts.)

Erweiterungen:

5c. Das BAS informiert das System über invalide Informationen auf der Karte.

5c.1. Das System informiert den Kunden und gibt die Karte aus.

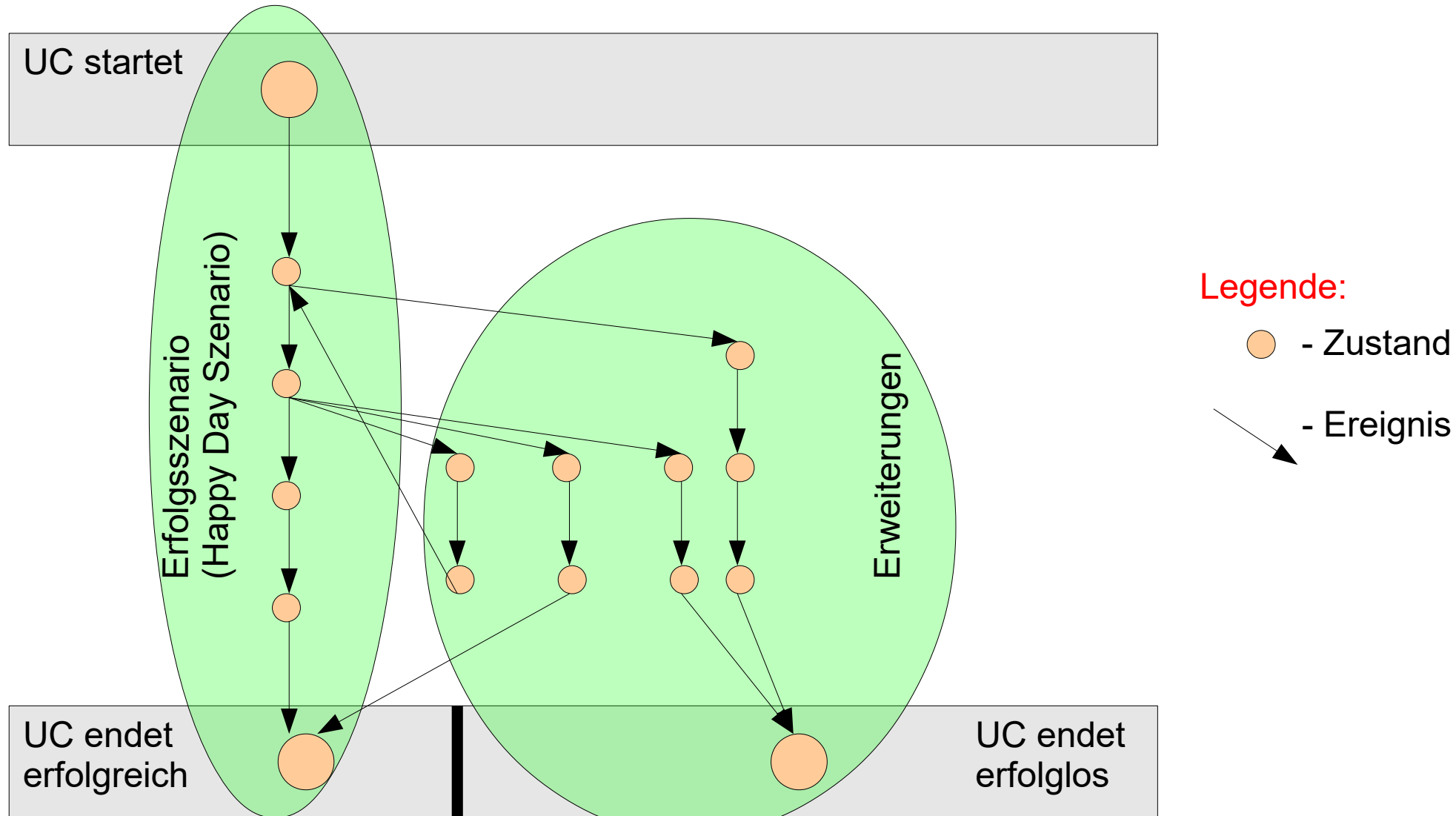
5c.2. Use Case endet erfolglos.

5d. Das System überschreitet beim Warten auf eine Antwort von dem BAS ein Zeitlimit.

5d.1 Das System informiert den Kunden dass es zur Zeit ausser Betrieb ist und gibt die Karte aus.

5d.2. Use Case endet erfolglos.

Verhältnis von Erfolgsszenario und Erweiterungen



Erfolg und Misserfolg

Ein Use Case ist in der Regel mit **erfolgreichen** und **erfolglosen Szenarien** beschrieben

- **Erfolgreiches Szenario**
 - Der Primäre Akteur hat **sein Ziel** erreicht
- **Erfolgloses Szenario**
 - Ziel wurde **nicht erreicht**, aber das System befindet sich in einem **konsistenten Zustand** gemäß formulierter **Nachbedingung** des Use Case

Vor- und Nachbedingung

- Vorbedingung

- Welche **Annahme** kann **beim Start** des Use Case getroffen werden?
 - *Verbindung zum Bank-System ist sichergestellt.*
- Der Use Case muss noch **durch ein Ereignis gestartet** werden (Eintreten der **Vorbedingung** ist nicht ausreichend!)

- Nachbedingung

- Cockburn nennt diese "minimale Garantien"
- Was kann **nach Ablauf jedes Szenarios** (nicht nur der erfolgreichen) **garantiert** werden?
- Sich fragen: "Was würde einen **Stakeholder unglücklich** machen?"
 - *Alle Transaktionen sind geloggt.*
 - *Der ausgezahlte Betrag ist genau der Betrag, der vom Konto abgebucht wird.*

Primärer Akteur

- Ist der Akteur ...
 - welcher Use Case gestartet hat
 - dessen Ziele erreicht werden sollen

Falls das Use Case durch Zeitereignis gestartet wird,
wird das System zum Primären Akteur.

Arten von Erweiterungen

Erweiterungen sind hauptsächlich bei folgenden Sachverhalten notwendig:

1) Es gibt einen alternativen Erfolgspfad

*Der Angestellte gibt direkt die Bestellnummer ein.
(anstatt nach der Bestellung zu blättern)*

2) Ausbleibende Aktion eines Aktors, z.B. Zeitüberschreitung bei Warten auf Antwort

Die Zeit zur Eingabe des Kundenpassworts ist überschritten.

3) Alternativer Pfad bei erfolgloser Validierung

Das System stellt fest, dass das eingegebene Passwort falsch ist.

4) Interner Fehler innerhalb des Systems (der erkannt werden kann)

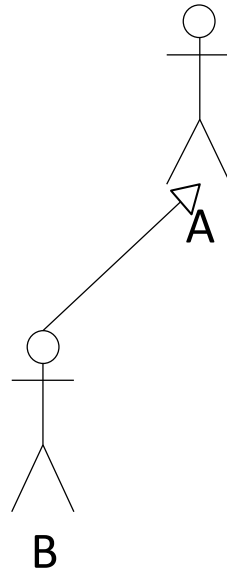
Das System stellt einen Papierstau im Drucker fest.

Formulierung um Ereignisfluss zu strukturieren

- *(1-5)a. (jederzeit in den Schritten 1-5)*
- *Use Case wird in Schritt 5 fortgesetzt.*
- *Die Schritte 1-4 können in beliebiger Reihenfolge ausgeführt werden.*
- *Die Schritte 1-4 sind optional.*
- *...*

Aktoren und Use Cases organisieren

Vererbung von Aktoren

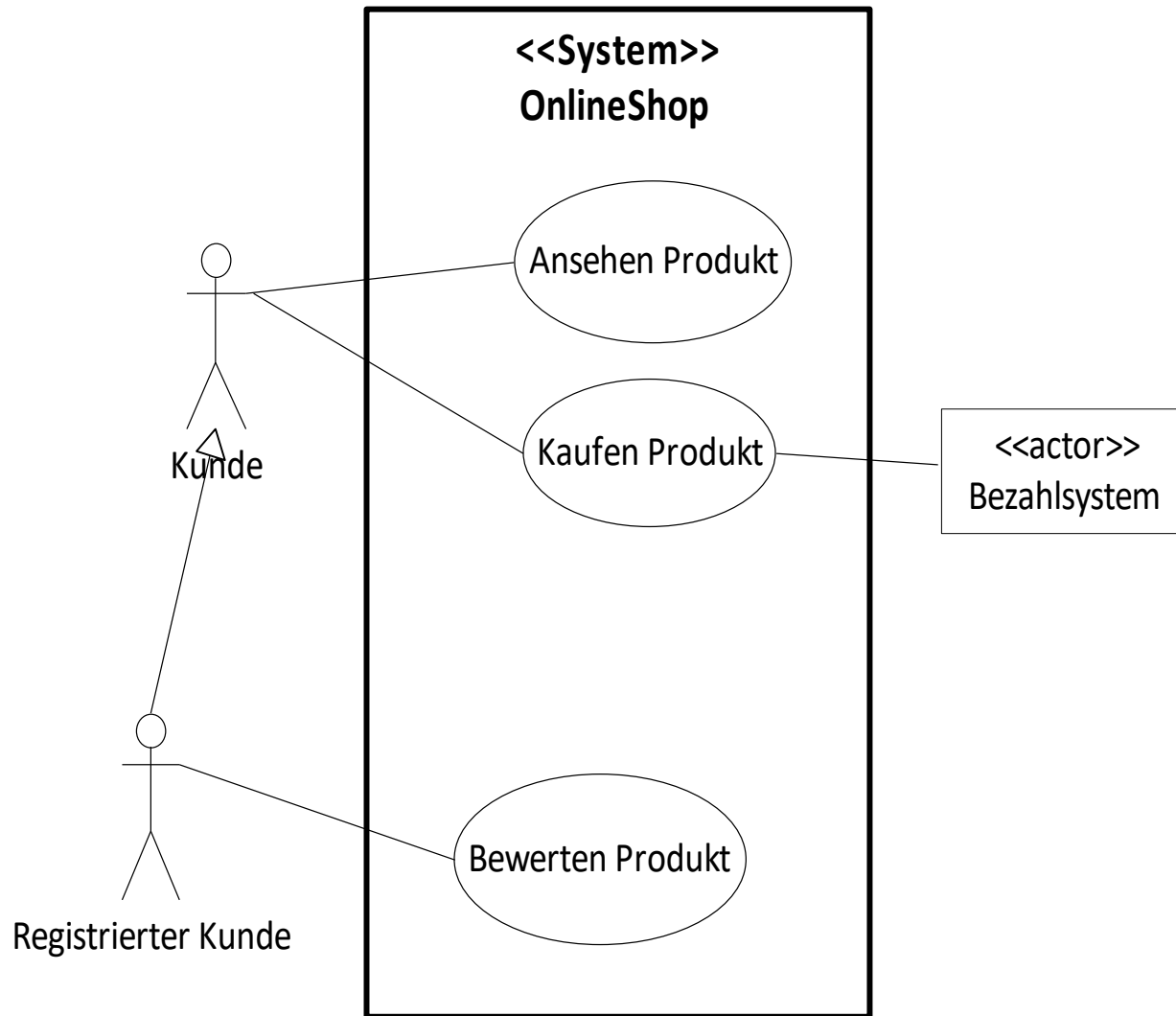


Lese:
B erbt von A

Konvention: Superaktor (A) wird oberhalb des Subaktors (B) platziert.

- **Aktor B** erbt alle Fähigkeiten von **Aktor A**
 - Insbesondere wird die Teilnahme an Use Cases, an denen A beteiligt ist, an B vererbt
- Anwendungen haben oft ein hierarchisches Konzept von Nutzerrollen

Beispiel Vererbung von Aktoren



<<include>> (<<uses>>)-Beziehung zwischen Use Cases

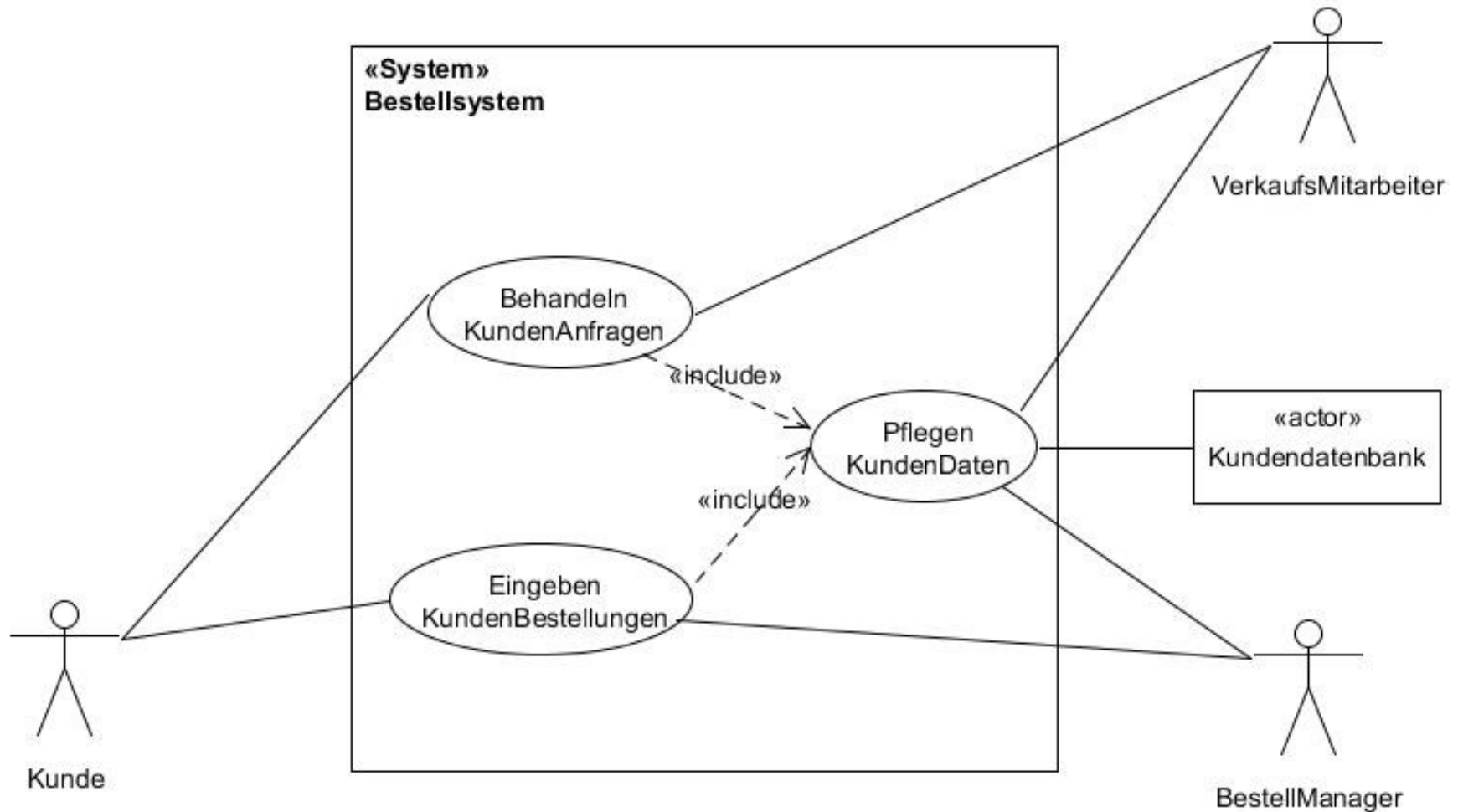


Lese:
A inkludiert B

Ein Schritt in A ruft den Use Case B auf

- A wird "inkludierendes" und B "inkludiertes" Use Case genannt
- In älteren Version der UML, die Beziehung wird mit Stereotyp <<uses>> anstatt mit Stereotyp <<include>> annotiert
- <<include>> wird benutzt, wenn Verhalten einmal beschrieben und mehrfach genutzt werden soll
 - Inkludiertes UC B sollte von mehr als einem inkludierenden UC A genutzt werden

Beispiel «include»



Beispiel <<include>>

Use Case: Behandeln KundenAnfragen

Kurzbeschreibung : Der Use Case erlaubt dem Kunden Anfragen zu stellen.

Primärer Akteur: Kunde

Erfolgsszenario:

1. Der Kunde ruft im Service-Center an.
2. Das System öffnet einen Customer Service Request (CSR) und protokolliert Datum und Uhrzeit.
3. Das System leitet den Anruf an einen verfügbaren VerkaufsMitarbeiter weiter und zeigt diesem den CSR an.
4. VerkaufsMitarbeiter gibt die Kunden-ID ein und pfllegt die KundenDaten. *

[Die restliche UseCase Beschreibung ist aus Platzgründen weggelassen.]

* -- der unterstrichene Text markiert das Inkludieren (Aufrufen/Benutzen) eines anderen Use Case

Beispiel <<include>>

Use Case: Pflegen KundenDaten

Kurzbeschreibung : Der Mitarbeiter (bestimmt durch das inkludierende Use Case) ändert die erfassten Informationen über einen Kunden.

Primärer Akteur: Mitarbeiter

Erfolgsszenario:

1. Der Mitarbeiter ändert oder erweitert Kunden-Informationen in beliebiger Reihenfolge.
2. Der Mitarbeiter speichert die geänderten und erweiterten Informationen.
3. Das System sendet die Daten zur Kunden Datenbank.
4. Use Case endet erfolgreich.

Beispiel <<include>>

Erweiterungen:

2a. Der Mitarbeiter **bricht** die Eingabe bzw. Änderung von Kunden Informationen **ab**.

2a.1. Use Case wird in Schritt 1 fortgesetzt.

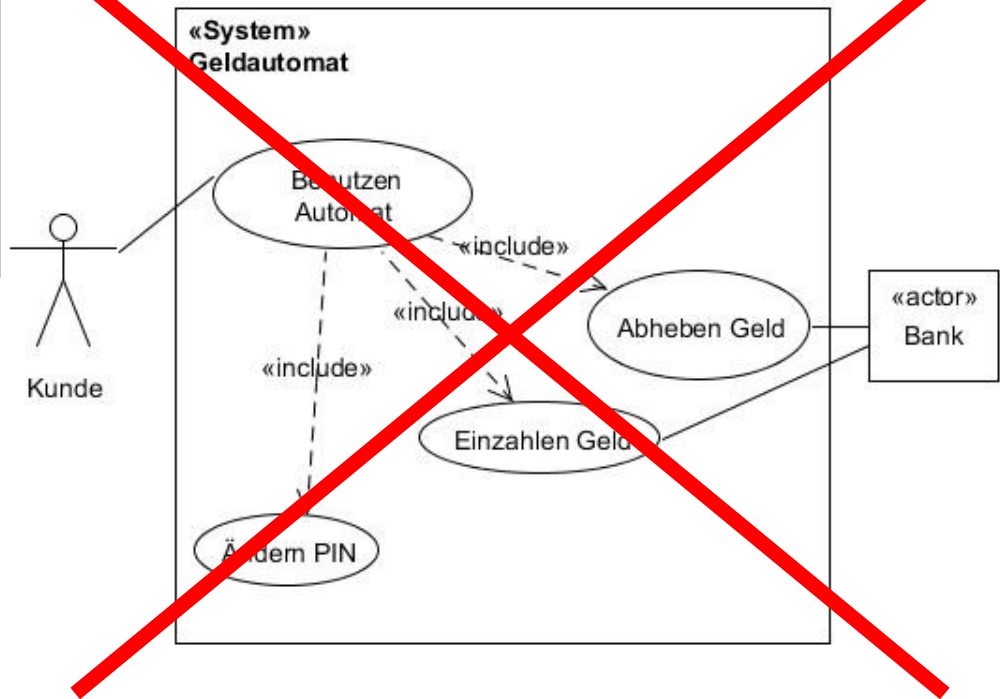
3a. Das System **stellt fest**, dass die Kunden Datenbank ausser Betrieb ist.

3a.1. Das System **informiert** den Mitarbeiter über die ausgefallene Datenbank.

3a.2. Use Case endet erfolglos.

Fallen bei der Benutzung von <<include>>

<<include>> sollte **niemals(!)** für funktionale Dekomposition missbraucht werden!



- Niemals ein inkludiertes Use Case einführen, solange man nicht zwei identische Abschnitte in der textuellen Beschreibung von zwei (inkludierenden) Use Cases gefunden hat!
- Das inkludierende Use Case nicht zur schnöden Zusammenfassung von UCs verwenden
 - Jedes Use Case repräsentiert ein Ziel des Primären Aktors!

Use Case und Projektplanung

Use Cases sind detaillierte Beschreibungen von Systemfunktionalitäten, die es zu implementieren gilt. Use Cases sind oftmals Grundlage der weiteren Projektplanung.

- Nutzer sollten alle Use Cases priorisieren
- SW Entwickler sollten die Komplexität der Umsetzung einschätzen

Grundregel: Risikoreiche, wichtige Use Case werden zuerst realisiert.

Use Case	Nutzerpriorisierung	Komplexität
Konto anlegen	mittel	einfach
Notarkonto anlegen	top	einfach
Geld auf Konto einzahlen	hoch	mittel
Geld von einem Konto auf anderes Konto überweisen	hoch	schwierig
Geld von Konto abheben	hoch	Mittel
Konto schließen	top	einfach

Use Cases - Abschließende Tipps

- **Erfolgsszenario zuerst** - Erst Erfolgsszenario beenden bevor mit Erweiterungen begonnen wird
- **Brainstorming mit allen Beteiligten** - Das Erfassen aller Erweiterungen ist sehr aufwändig. Eine bewährte Technik sind Stakeholder-Interviews, um optionales Verhalten sowie Ausnahme- und Fehlersituationen zu identifizieren.
- **Bedingungen zuerst notieren:** Bevor das Verhalten im Erweiterungsfälle formuliert wird, sollte eine komplette Liste von Ausnahmebedingungen erstellt werden ("Was kann schief gehen?").
- **Permanentes Umschreiben** - Gute Use Case Beschreibungen sind Ergebnis von vielen Iterationen.

Use Cases - Abschließende Tipps

- **Schreibstil festlegen** -
 - Alistair Cockburn hat 18 Schreibstile identifiziert
 - **Passen** Sie sich an den **Schreibstil** ihrer Firma **an** (oder umgekehrt !)
- **Richtige Granularität** - **Anzahl** der Use Cases ist projektspezifisch
 - 20-40 sind eine bewährte Größe
- **Breite zuerst** - **Nicht** mit zu detaillierten Beschreibungen beim ersten Projekttreffen aufwarten (werden nicht mehr diskutiert)
- **Vollständigkeit** - Use Cases sollten am **Schluss** detailliert sein
 - **Funktionalität** muss der **Auftraggeber** festlegen, nicht der **Entwickler** während der Implementierung !!!

Story – Beispiel AbhebenGeld

Erklären Sie, warum die untere UC-Beschreibung mangelhaft ist. Welche wichtigen Informationen fehlen? Wie würden Sie die UC Beschreibung umformulieren?

AbhebenGeld (Happy Day Szenario)

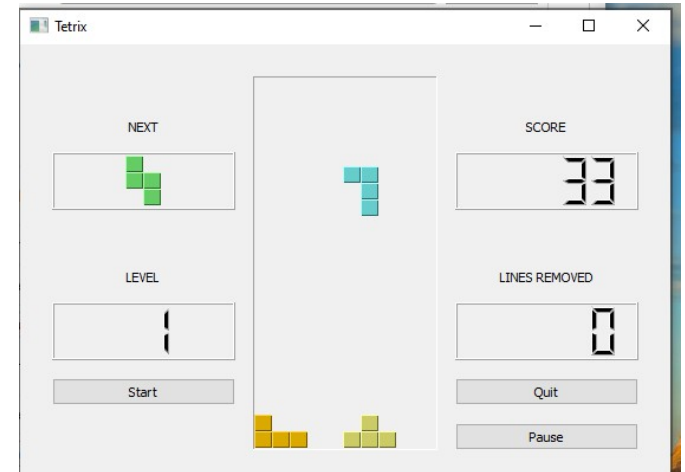
- 1) Das Use Case startet, sobald der Kunde die Bankkarte in den Automaten einsteckt.
- 2) Das System liest die Bankkarte und fordert den Kunden zur Eingabe der Personal Identification Number (PIN) auf.
- 3) Das System bietet ein Auswahlmenu an.
- 4) Der Kunde wählt das Abheben von Geld.
- 5) Das System fragt den abzuhebenden Betrag nach und der Kunde sollte den Betrag eingeben.
- 6) Geldnoten in Höhe des eingegebenen Betrags sowie die Karte werden ausgegeben.
- 7) Der Kunde wird das Geld und die Karte nehmen.
- 8) Das Use Case endet erfolgreich.

Registrierungssystem

- Finden Sie Aktoren und Ziele in der folgenden informellen Beschreibung:
 - *Eine Universität möchte ein Registrierungssystem einführen, mit dem sich Studierende für ihre Kurse registrieren können. Die Liste der aktuellen Kurse kann nur von autorisierten Personen geändert werden.*
- Erarbeiten Sie eine textuelle Beschreibung für das Use Case *ÄndernKursRaum*. Dieses Use Case soll das Verlegen eines Kurses in einen anderen Raum ermöglichen. Beachten Sie, dass es bei der Verlegung zu keinen Konflikten mit anderen Kursen kommen darf (zu jeder Zeit darf in einem Raum immer nur ein Kurs stattfinden).

Beschreibung Tetris

In der Qt Distribution (auch in Community-Version) finden sich viele wundervolle Programmier-Beispiele, unter anderem das Projekt "Tetrix"



- Erstellen Sie eine textuelle UC-Beschreibung für den UseCase "Spielen", mit der die Regeln des Tetris-Spiels klar werden
 - Anmerkung: Für diese Aufgabe brauchen Sie nicht in den Code zu schauen
- Schauen Sie in den Implementierungscode und versuchen Sie, diesen Code zu "visualisieren". Welche Aspekte des Codes werden durch Ihre Visualisierung erfasst? Den Code finden Sie auf Moodle.